PROJECT REPORT ON

SiteScan

Passive Web Security Posture Assessment and Vulnerability Reporting Tool

SUBMITTED BY

Harisudhan S
RA2411030020081

Tharun Suresh
RA2411030020084

SUBJECT

21CSE382T / Security Management

SiteScan

Abstract

SiteScan is a web security assessment tool designed to help developers, students, and security analysts evaluate the security posture of a website quickly and safely. Misconfigured web servers remain one of the most common entry points for data breaches, session hijacking, and supply-chain compromise, because a single missing header, an exposed *.env* file, or an outdated JavaScript library can undo otherwise sound application code. SiteScan accepts a target URL and evaluates the live responses for security signals such as weak Content-Security-Policy rules, deprecated TLS protocols, insecure cookies, version-revealing banners, publicly reachable secrets, and missing email-authentication records.

The platform is organised into nine independent scanner modules covering HTTP security headers, TLS/SSL, cookies, information disclosure, sensitive-path exposure, transport hygiene, DNS and email posture, front-end dependency vulnerabilities, and robots.txt / sitemap.xml / security.txt. The modules are discovered automatically through a registry decorator and run concurrently under an asynchronous orchestrator that enforces a global rate limit and permits only safe HTTP methods. Detected library versions are checked against the OSV.dev vulnerability database for known CVEs.

Results are presented as a weighted 0-100 posture score with a letter grade, and every finding carries a severity, CVSS 3.1 vector, raw evidence, OWASP Top 10 2021 category, CWE identifier, and remediation advice. Reports are produced as a colour-coded terminal table, machine-readable JSON, and a self-contained HTML report, and the exit code allows the tool to act as a CI/CD security gate. SiteScan is strictly passive: it performs no exploitation, brute force, or fuzzing, and it should be used only on websites the operator is authorised to test.

Architecture Diagram

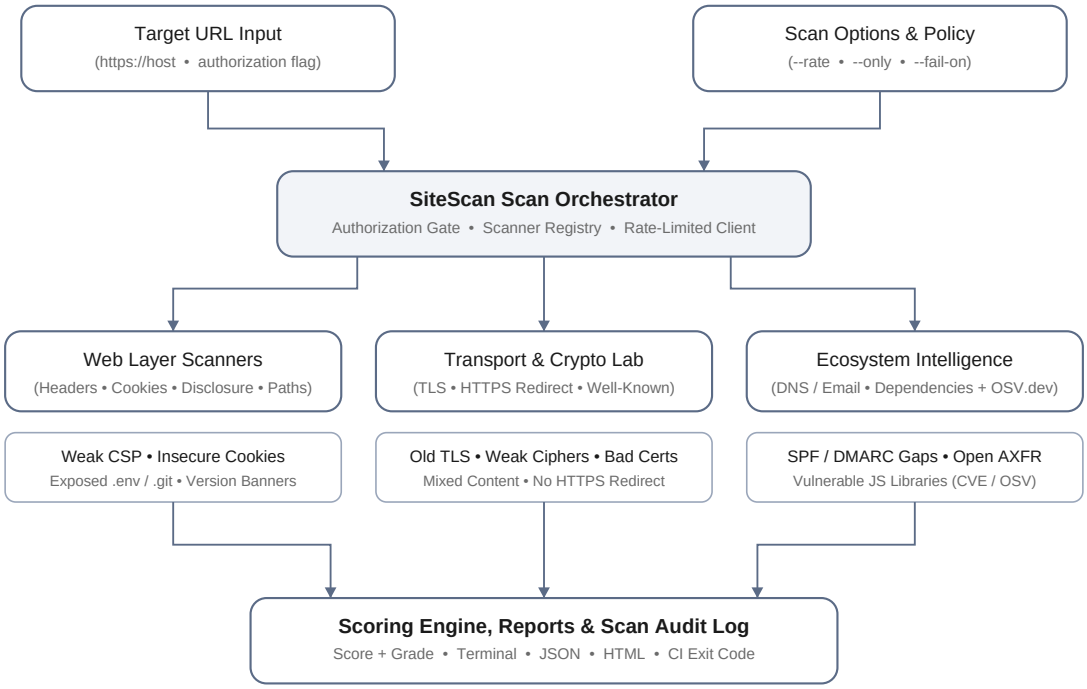

Figure 1. High-level architecture of SiteScan.

SiteScan — Project Explanation

Explanation of the Project

1. Input and authorization — The user supplies the target URL together with the mandatory *--i-am-authorized* flag and then types the exact hostname to confirm. Targets on a built-in denylist of major third-party platforms are refused before any request is sent, and every scan is logged to *scans/<timestamp>-<host>.json*.

2. Orchestration and scanner registry — The Python asyncio orchestrator discovers every scanner module through a *@register* decorator, runs them concurrently, and shares a single rate-limited HTTP client (5 requests per second by default) that allows only GET, HEAD, and OPTIONS. Timeouts, connection errors, and redirect loops are contained so that one failing check never stops the scan.

3. Web-layer analysis — Responses are examined for missing or weak security headers (CSP, HSTS, X-Frame-Options, Referrer-Policy, Permissions-Policy), cookies without Secure, HttpOnly, or SameSite, verbose error pages, directory listings, and well-known sensitive paths such as */.git/config* and */.env*. Paths are probed with HEAD requests only, so their contents are never downloaded.

4. Transport and cryptographic analysis — TLS handshakes are attempted with each protocol version and with weak cipher families to identify deprecated protocols and insecure suites. The certificate is checked for expiry, self-signing, hostname mismatch, weak keys, and chain validity, and the site is checked for HTTP-to-HTTPS redirection, mixed content, and HSTS preload eligibility.

5. Ecosystem intelligence — DNS records are evaluated for SPF, DKIM, DMARC policy strength, CAA, and open zone transfers. Front-end libraries such as jQuery, Bootstrap, and AngularJS are fingerprinted from script URLs and banners, and their versions are queried against OSV.dev to list matching CVEs.

6. Scoring, reporting and audit — Findings are de-duplicated, sorted by severity, and converted into a weighted posture score and grade. The terminal table, JSON output, and HTML report (executive summary, severity breakdown chart, and per-finding cards) present the results, and a non-zero exit code is returned when High or Critical issues are found so that CI pipelines can block insecure releases.

Security Checks Performed

Each scanner module targets a specific class of misconfiguration and reports the indicators below, mapped to OWASP Top 10 2021 categories and CWE identifiers.

Module	What It Inspects	Typical Red Flags
HTTP Security Headers	CSP, HSTS, framing, MIME, referrer and permissions policies	unsafe-inline / unsafe-eval in CSP, wildcard script sources, missing HSTS or X-Frame-Options
TLS / SSL	Protocol versions, cipher suites, certificate and chain	TLS 1.0/1.1 enabled, anonymous or static-RSA ciphers, self-signed, expiring, weak RSA key
Cookies	Attributes of every Set-Cookie header	Session cookie without Secure or HttpOnly, missing SameSite, SameSite=None without Secure
Information Disclosure	Version banners, error pages, directory indexes	Server: Apache/2.4.29, X-Powered-By: PHP/7.2.3, stack traces, “Index of /”
Sensitive Paths	HEAD-only probe of ~35 well-known paths	/.env, /.git/config, /backup.sql, /phpinfo.php returning HTTP 200
Transport Hygiene	HTTPS redirect, mixed content, HSTS preload	No 301 to HTTPS, http:// scripts or iframes on HTTPS pages
DNS / Email Posture	SPF, DKIM, DMARC, CAA, zone transfer	SPF +all, DMARC p=none, no CAA record, AXFR permitted
Dependency Surface	JS/CSS library versions checked on OSV.dev	jQuery 1.12.4 (CVE-2020-11022), AngularJS 1.5.8, Bootstrap 3.3.7
robots / sitemap / security.txt	Crawler files and RFC 9116 disclosure policy	robots.txt revealing /admin/ or /backup/, missing or expired security.txt

Technologies Used

- Python 3.11+ with asyncio as the core orchestrator, scanner runtime, and report synthesis engine.
- httpx (async) for rate-limited HTTP; Python ssl and the cryptography library for TLS and X.509 analysis.
- dnspython for SPF, DKIM, DMARC, CAA, and AXFR checks; the OSV.dev API for dependency CVE lookups.
- rich for the colour-coded terminal report and Jinja2 for the self-contained HTML report.
- pytest with a local Flask target application for offline, repeatable positive and negative testing.

How to Run

Install once inside a virtual environment, then scan a site you are authorised to test:

```
python -m venv .venv && .venv\Scripts\activate
pip install -r requirements.txt && pip install -e .
sitescan https://your-site.example --i-am-authorized \
    --html report.html --json report.json
```

The tool asks the operator to retype the hostname, runs all nine checks, prints the table, and writes the reports. *python scripts/generate_sample.py* produces an offline demonstration report against the bundled practice site, and *pytest* runs the 61-test suite.

Advantages

- Covers nine categories of web misconfiguration in a single run that takes seconds.
- Strictly passive and rate-limited, so it is safe to run against production and staging systems.
- Each finding includes evidence, CVSS, OWASP and CWE mapping, and concrete remediation guidance.
- Exit codes and JSON output make it straightforward to use as an automated CI/CD security gate.
- Modular registry design means a new check can be added as a single file without changing other code.
- Self-contained HTML report can be shared with management and developers without extra tooling.

Applications

- Pre-release security checks in development pipelines and staging environments.
- Periodic posture monitoring of an organisation’s own public websites by IT or SOC teams.
- Security awareness and teaching of common web misconfigurations using the bundled practice site.
- Preparing a first-pass findings list before an authorised manual penetration test.

Limitations and Responsible Use

Automated checks may produce false positives or false negatives; for example, DKIM can only be verified for common selector names, and libraries bundled without version banners cannot be fingerprinted. Because SiteScan is passive, it does not test business logic, authentication flows, or injection flaws, so a clean report is not proof that a site is secure and is not a substitute for a manual penetration test or source code review. Scanning systems without permission may be illegal, so the tool must be used only on websites the operator owns or has written authorisation to assess.

Conclusion

SiteScan provides a structured and accessible first line of defence against web server misconfiguration by combining header and cookie analysis, TLS inspection, exposure detection, DNS and email posture checks, and dependency vulnerability lookup within a single, safe, and extensible tool. It helps developers and analysts identify and prioritise weaknesses quickly through clear scoring and standards-based reporting. Its findings should always be verified manually and used alongside conventional security controls such as secure configuration baselines, patch management, web application firewalls, and regular penetration testing.